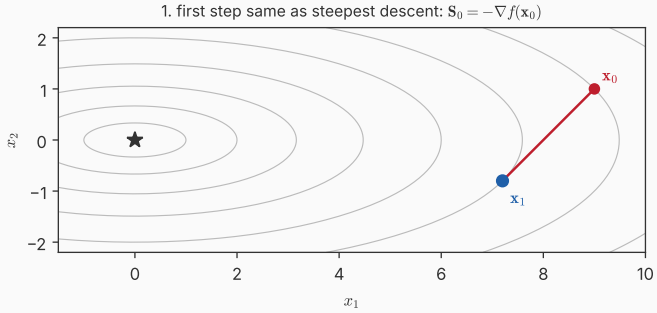
Optimization

3.3.3 Conjugate gradient method (Fletcher–Reeves)

José Firmino Aguilar Madeira

The same start — where steepest descent would begin to zigzag

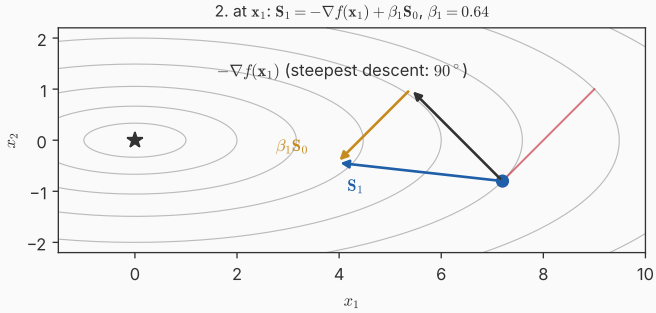
The example from 3.3.1: $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ from $\mathbf{x}_0 = (9, 1)$, exact line search. The first step is the same as in steepest descent: $\mathbf{S}_0 = -\nabla f(\mathbf{x}_0)$, and we reach $\mathbf{x}_1 = (7.2, -0.8)$.



At $\mathbf{x}_1$, the gradient would turn 90° and the zigzag would begin (3.3.1). The question: is there a better direction, using only gradients?

Remember the previous direction

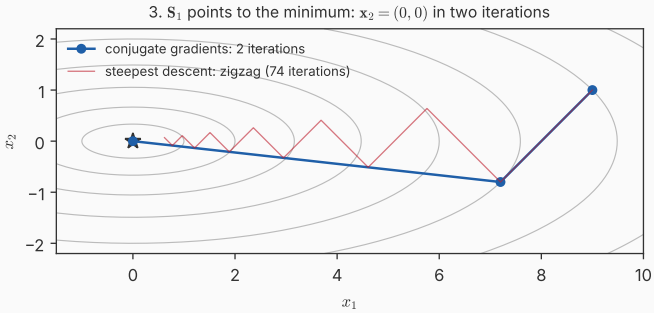
Instead of following only $-\nabla f(\mathbf{x}_1)$, we add a part of the previous direction: $\mathbf{S}_1 = -\nabla f(\mathbf{x}_1) + \beta_1 \mathbf{S}_0$, with $\beta_1 = \dfrac{\|\nabla f(\mathbf{x}_1)\|^2}{\|\nabla f(\mathbf{x}_0)\|^2} = \dfrac{103.68}{162} = 0.64$ (Fletcher–Reeves).



Extra cost: no additional gradient — only storing $\mathbf{S}_0$ and $\|\nabla f(\mathbf{x}_0)\|^2$, and two inner products.

Two iterations instead of 74

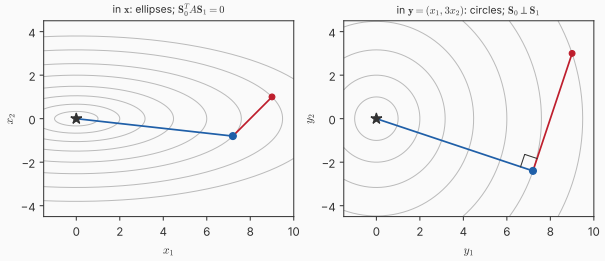
With $\mathbf{S}_1 = (-12.96,\ 1.44)$ and exact line search ($\lambda_1^* = 5/9$): $\mathbf{x}_2 = (0, 0)$, the minimum, in two iterations ($n = 2$).



Steepest descent needed 74 iterations to reach $\|\nabla f\| < 10^{-6}$, with the same information per iteration.

What does “conjugate” mean?

$\mathbf{S}_0$ and $\mathbf{S}_1$ are not perpendicular, but they are *A*-conjugate, with $A = \nabla^2 f = \mathrm{diag}(1, 9)$: $\mathbf{S}_0^T A\, \mathbf{S}_1 = (-9, -9)\,(-12.96,\ 12.96)^T = 116.64 - 116.64 = 0$. Changing the scale, $y_1 = x_1$, $y_2 = 3x_2$, the ellipses become circles — and the conjugate directions become perpendicular.



With circles, two perpendicular steps with exact search reach the centre. “Conjugate” = perpendicular in the geometry of *A*: minimizing along $\mathbf{S}_1$ does not spoil what was gained along $\mathbf{S}_0$.

Now formally

Quadratic $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$ with $A = A^T \succ 0$ (SPD). *A*-conjugate directions:

$$\mathbf{S}_i^T A\,\mathbf{S}_j = 0 \qquad (i \neq j) \qquad \text{(in the example: } \mathbf{S}_0^T A\,\mathbf{S}_1 = 0\text{)}.$$

Result

SPD quadratic + exact arithmetic + exact line searches along *A*-conjugate directions $\Rightarrow$ the minimum in at most n iterations.

Fletcher and Reeves (1964) generate these directions *using only gradients* ($\nabla f_k = \nabla f(\mathbf{x}_k)$):

$$\mathbf{S}_0 = -\nabla f_0, \qquad \mathbf{S}_k = -\nabla f_k + \beta_k \mathbf{S}_{k-1}, \qquad \beta_k^{FR} = \frac{\nabla f_k{}^T \nabla f_k}{\nabla f_{k-1}{}^T \nabla f_{k-1}} = \frac{\|\nabla f_k\|^2}{\|\nabla f_{k-1}\|^2}.$$

Steepest descent (3.3.1): consecutive gradients orthogonal, $\nabla f_{k+1}^T \nabla f_k = 0$

Conjugate gradients: *A*-conjugate directions, $\mathbf{S}_i^T A\mathbf{S}_j = 0$: orthogonal "in the geometry of *A*"

The algorithm (Fletcher–Reeves)

Fletcher–Reeves: $\mathbf{x}_0$, ε

Step 1 Choose the starting point $\mathbf{x}_0$ and $\varepsilon > 0$.

Step 2 First direction: set $\mathbf{S}_0 = -\nabla f_0$.

Step 3 Set $\mathbf{x}_1 = \mathbf{x}_0 + \lambda_0^* \mathbf{S}_0$ (λ_0^* optimal along $\mathbf{S}_0$).
If $\|\nabla f_1\| \leq \varepsilon$, **stop**; otherwise, set $k = 1$ and go to step 4.

Step 4 Set $\mathbf{S}_k = -\nabla f_k + \dfrac{\|\nabla f_k\|^2}{\|\nabla f_{k-1}\|^2} \mathbf{S}_{k-1}$.

Step 5 Compute λ_k^* (optimal along $\mathbf{S}_k$);
set $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k^* \mathbf{S}_k$.

Step 6 If $\|\nabla f_{k+1}\| \leq \varepsilon$, **stop**.
Otherwise, set $k = k + 1$ and go back to step 4.

current gradient + memory of the previous direction

- Cost per iteration: $n_g = 1$; $n_f =$ evaluations of f in the line search; plus two inner products.
- Memory: $\mathbf{x}_k$, ∇f_k, $\mathbf{S}_k$ — $O(n)$, no Hessian.
- On the SPD quadratic, with exact search and exact arithmetic: at most n iterations. And beyond quadratics?

Beyond quadratics: why and when to restart

Restart = set $\beta_{k+1} = 0$, that is, go back to $\mathbf{S}_{k+1} = -\nabla f_{k+1}$. Why: conjugacy was built for a quadratic with exact search; on a non-quadratic f, or with inexact search, it is gradually lost and the "memory" starts to get in the way.

When to restart	Test	Why
1. Every n iterations	$(k+1) \bmod n = 0$	on a quadratic n directions suffice
2. $\mathbf{S}_{k+1}$ is no longer a descent direction	$\nabla f_{k+1}^T \mathbf{S}_{k+1} \geq 0$	the line search would not decrease f
3. Consecutive gradients far from orthogonal	see note*	conjugacy has been lost

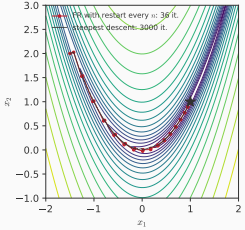
* Powell's criterion (1977): restart if $|\nabla f_{k+1}^T \nabla f_k| \geq \nu \left\| \nabla f_{k+1} \right\|^2$, with $\nu \approx 0.1$–0.2 (with exact search on a quadratic, $\nabla f_{k+1}^T \nabla f_k = 0$).

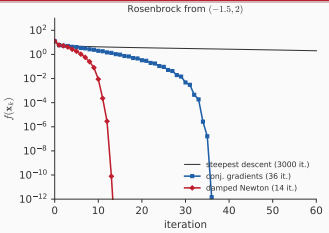
Effect on Rosenbrock (next page): FR without restart, 102 iterations; with restart every $n = 2$, 36. The MATLAB code in these notes uses tests 1 and 2.

Another common choice of β: Polak–Ribière$^+$

$\beta_k^{PR+} = \max\left(0,\ \nabla f_k^T (\nabla f_k - \nabla f_{k-1}) / \left\| \nabla f_{k-1} \right\|^2\right)$ — beyond quadratics, it restarts by itself ($\beta \geq 0$): when the fraction is negative, $\beta = 0$.

Rosenbrock: steepest descent vs. conjugate gradients vs. Newton





- Rosenbrock is not quadratic, but nearly so near the minimum: FR with restart every $n = 2$ iterations converges in 36 iterations ($\|\nabla f\| < 10^{-4}$); without restart, 102; steepest descent does not converge in 3000.
- Damped Newton: 14 iterations — but each one costs a Hessian. Counted until $f < 10^{-4}$, with the same high-precision line search (Brent, tol. 10^{-10}): Newton $n_g = n_H = 12$, $n_f = 269$ in the line search (281 with f at the iterates); FR $n_g = 34$, $n_f = 684$ (718). For $n = 2$ the line search is the largest cost; as n grows, forming, storing and factorizing the Hessian makes gradient-only methods ever more attractive.

number of iterations $\neq$ cost

On the computer

The core, in MATLAB:

```
g = grad(x);  S = -g;      % 1st direction
% ... at each iteration:
lam = ...;                 % line search
x  = x + lam*S;
gn = grad(x);
beta = (gn'*gn)/(g'*g);    % Fletcher-Reeves
S  = -gn + beta*S;
if mod(k+1,n)==0 || gn'*S >= 0
    S = -gn;               % restart
end
g = gn;
```

The full function (`ConjGrad`, with `info.nfev`, `info.ngev`) is in the course companion code. On the quadratic: 2 iterations, $n_g = 3$.

The line search is not a detail

On the quadratic, exact conjugacy presupposes exact line search ($\nabla f_{k+1}^T \mathbf{S}_k = 0$). On nonlinear problems one normally uses *inexact* searches that are sufficiently accurate (Wolfe conditions), together with restarts.

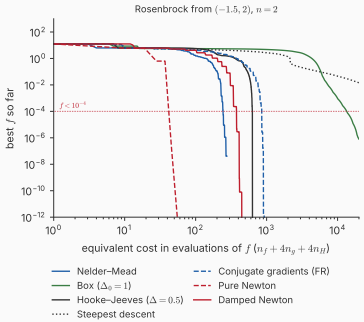
With `fminbnd` at its default tolerance, FR on Rosenbrock may fail to converge: of the three methods studied here, it is particularly sensitive to the quality of the line search.

The methods studied on the same problem, with the cost in evaluations

Rosenbrock from $(-1.5, 2)$; stopping: first evaluation (or iterate) with $f < 10^{-4}$ (f at the iterates counts in n_f); in 3.3, accurate line search (Brent, tol. 10^{-10}). The grid (3.2.2), for exploring and initializing, is left out.

Method	Derivatives (order)	n_f	n_g	n_H	equiv. in f
Localized random†	0	975 [867, 1052]	–	–	975
Nelder–Mead*	0	244	–	–	244
Box ($\Delta_0 = 1$)	0	13 445	–	–	13 445
Hooke–Jeeves* ($\Delta = 0.5$)	0	627	–	–	627
Steepest descent‡	1	60 900	3000	–	72 900
Conj. grad. (FR, restart n)	1	718	34	–	854
Pure Newton	2	6	5	5	46
Damped Newton	2	281	12	12	377

* algorithms of 3.2.3 and 3.2.5; with the standard versions (no $f(\mathbf{x}_c)$; HJ of 1961): 166 and 430. † stochastic: median [quartiles] of 30 runs. ‡ does not reach $f < 10^{-4}$ in 3000 iterations ($f = 2.4 \cdot 10^{-4}$). Equiv. in f (central derivatives, $n = 2$): $n_f + 4n_g + 4n_H$ (the Hessian, $2n^2 + 1 = 9$ points, reuses the 4 of the gradient and $f(\mathbf{x}_k)$).



On this problem, with these parameters: pure Newton has the lowest cost when it converges to the intended solution; Nelder–Mead beats the first-order methods because for $n = 2$ the accurate line search costs ≈ 20 evaluations per iteration. With large n ($n = 100$: numerical gradient 200, Hessian 20 001) gradient-only methods gain ground — there is no winner in the abstract.

Summary table — 3.3

What information do we have about f? How much does it cost to obtain it? What can we guarantee with it?

Method	Information	Cost per iteration	Guarantee	Main limitation
Steepest desc. (3.3.1)	∇f	$n_g = 1$; $n_f =$ cost of the line search; algebra $O(n)$	stationarity from any $\mathbf{x}_0$ (bounded level set, Lipschitz gradient, suitable search); linear, bound $((\kappa-1)/(\kappa+1))^2$ (SPD quadr., exact search)	zigzag if κ large
Newton (3.3.2)	∇f, $\nabla^2 f$	$n_g = n_H = 1 + n \times n$ system ($O(n^3)$); memory $O(n^2)$)	local quadratic if $H(\mathbf{x}^*) \succ 0$ (under regularity; damped: $\lambda_k \to 1$ with Armijo/Wolfe from $\lambda = 1$); 1 it. on SPD quadratics	expensive Hessian; without damping it may diverge or go to maxima/saddles
Conj. grad. (3.3.3)	∇f + previous direction	$n_g = 1$; $n_f =$ cost of the line search; memory $O(n)$	$\leq n$ it. on SPD quadratics (exact search and arithmetic); in general, $\liminf \|\nabla f(\mathbf{x}_k)\| = 0$ under the row-1 assumptions with exact or strong Wolfe search (Al-Baali, 1985)	sensitive to the line search

Rosenbrock, accurate line search, stop at $\|\nabla f\| < 10^{-4}$: steepest descent 3000 it. (not reached; $n_g = 3000$); damped Newton 14 it. ($n_g = 15$, $n_H = 14$); FR with restart 36 it. ($n_g = 37$). Steepest descent if n is huge and accuracy modest; Newton if the Hessian is cheap; conjugate gradients and quasi-Newton balance cost per iteration and use of curvature/memory.

KEY TAKEAWAYS — 3.3.3

1. Conjugate gradients = current gradient + memory of the previous direction: $\mathbf{S}_k = -\nabla f_k + \beta_k \mathbf{S}_{k-1}$ (Fletcher–Reeves).
2. A-conjugate directions: on an SPD quadratic, with exact search and exact arithmetic, the minimum in at most n iterations.
3. Cost per iteration almost equal to that of steepest descent, memory $O(n)$, no Hessian: an attractive compromise for large n.
4. Beyond quadratics: restarts and an accurate line search — of the three methods, it is the most sensitive to it.
5. Number of iterations $\neq$ cost: compare in evaluations, on this problem, with these parameters.

KEY TAKEAWAYS — Chapter 3

1. In several dimensions, in direction-based methods, each iteration chooses a direction and a step; the methods differ in the information they use.
2. Derivative-free (3.2): only values of f, to explore or to refine locally. NM has no general guarantee; HJ is at the origin of pattern search, whose formalized versions (GPS) have, under suitable assumptions, $\liminf \|\nabla f(\mathbf{x}_k)\| = 0$.
3. Derivative-based (3.3): steepest descent is cheap but slow if ill-conditioned; Newton is fast near the minimum, but expensive and needs safeguards; conjugate gradients sit in between.
4. Cost is counted in evaluations, not iterations; there is no winner in the abstract.
5. Local methods guarantee, at best, stationarity — not the global minimum.

Next (Ch. 4): and what if not all directions are allowed?

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Fletcher, R., Reeves, C. M., *Function minimization by conjugate gradients*, The Computer Journal 7(2):149–154, 1964.
- Powell, M. J. D., *Restart procedures for the conjugate gradient method*, Mathematical Programming 12(1):241–254, 1977.
- Al-Baali, M., *Descent property and global convergence of the Fletcher–Reeves method with inexact line search*, IMA Journal of Numerical Analysis 5(1):121–124, 1985.
- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2nd ed., Springer, 2006 — Ch. 5.

Resources

- Companion code: `ConjGrad` (`info.nfev`, `info.ngev`). MATLAB: `fminbnd`.
- Notebook `cap3_comparacao.ipynb`: reproduces the comparison table at the end of the chapter.